

8-Queen Problem Using Python

Aim

To write a Python program to solve the **8-Queen Problem** using the **Backtracking Algorithm**, such that no two queens attack each other on an 8×8 chessboard.

Algorithm

1. Start the program.
2. Create an empty 8×8 chessboard.
3. Place a queen in the first column.
4. Check whether placing the queen is safe (row, upper diagonal, and lower diagonal).
5. If the position is safe, place the queen and move to the next column.
6. If no safe position is found, backtrack and try another position.
7. Repeat the process until all 8 queens are placed.
8. Display the final chessboard configuration.
9. Stop the program.

Python Code

```
N = 8

board = [[0] * N for _ in range(N)]

def safe(row, col):
    for i in range(col):
        if board[row][i]:
            return False

    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j]:
            return False
        i -= 1
        j -= 1

    i, j = row, col
    while i < N and j >= 0:
        if board[i][j]:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
```

```

        return True

    for i in range(N):
        if safe(i, col):
            board[i][col] = 1

            if solve(col + 1):
                return True

            board[i][col] = 0

    return False

solve(0)

for row in board:
    print(row)

```

The screenshot shows a Python IDE with a file named 'EXP 2(8 Queen Problem).py'. The code defines a function 'safe' to check if a queen can be placed at a given row and column, and a recursive function 'solve' to find all possible solutions. The main code calls 'solve(0)' and prints the resulting board.

```

1  N = 8
2  board = [[0]*N for _ in range(N)]
3  def safe(row, col):
4      for i in range(col):
5          if board[row][i]:
6              return False
7      i, j = row, col
8      while i >= 0 and j >= 0:
9          if board[i][j]:
10             return False
11             i -= 1; j -= 1
12      i, j = row, col
13      while i < N and j >= 0:
14          if board[i][j]:
15              return False
16              i += 1; j -= 1
17      return True
18  def solve(col):
19      if col >= N:
20          return True
21      for i in range(N):
22          if safe(i, col):
23              board[i][col] = 1
24              if solve(col + 1):
25                  return True
26              board[i][col] = 0
27      return False
28  solve(0)
29  for row in board:
30      print(row)

```

The terminal output shows the following board configurations:

```

[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

```